Vigil: A Deterministic-First Runtime Firewall for Autonomous AI Agents with Graph-Native Context

Project Vigil

HackHydra Hackathon — Enterprise Ontology, Supply-Chain Blast Radius & Memory tracks
August 20, 2026 · Deployment: `vigil-hydra.onrender.com` · HydraDB database `vigil-os`
Source: `github.com/Aaditya1273/Vigil`

Abstract

Every widely deployed agent framework treats a tool call as an instruction to execute; the model that emitted it is also, implicitly, the authority that approved it. We present *Vigil*, a runtime firewall that interposes on every tool call an autonomous agent makes and decides — allow, pause, or block — through a pipeline in which deterministic checks always run, a knowledge graph is consulted when they are uncertain, and a language model is consulted last, if at all. The pipeline's central invariant is structural rather than procedural: the code path by which a model verdict could relax a deterministic block does not exist, and the model router returns an empty role set for unremarkable calls, so the happy path performs zero inference. Context the deterministic layer cannot hold — who a share recipient resolves to across aliases, the transitive blast radius of a package install, whether a behavioral pattern has precedent — is drawn from HydraDB, a managed graph store that extracts entities and relationships from ingested plain text; the graph is extracted, never hand-built. Inference, when reached, runs over a failover chain of three OpenAI-compatible vendors that retires a vendor only when it refuses, so an expired credit moves traffic rather than halting governance. Every decision, including every allow, lands in a SHA-256 hash-chained audit ledger and is mirrored into the graph for structural queries over the audit trail itself. We formalize the decision calculus, give the authority analysis showing which components can produce which outcomes, and report evaluation against real corpora: 95% retrieval recall on LongMemEval oracle questions, a live entity-resolution block of a customer-directed data share, and eleven distinct defects in our own integration found only by testing against the live services rather than their documentation. The reference implementation is 17,022 lines of Go with 129 passing tests, of which four suites run against the live graph and live inference vendors.

Keywords: autonomous agents; runtime governance; tool-call interception; knowledge graphs; HydraDB; MCP; fail-closed design; audit ledgers; LLM failover.

1 Introduction

An autonomous agent is a loop that converts model output into side effects: file writes, shell commands, network requests, purchases. Frameworks meter and observe this loop, but almost none of them *adjudicate* it — the same model that decided to run `pip install reqeusts` is the only authority that ever considered whether it should. Guardrail systems interpose on the conversation; observability platforms record the consequences; neither sits on the tool-call path with the power to refuse.

The objections to putting a model on that path are well known and correct. A model check on every call multiplies latency and cost by the call volume of the agent fleet. A model verdict is nondeterministic, so identical calls receive different treatment. And a model asked to police a model inherits the injection surface of the thing it polices. Vigil's position is that these are objections to *model-first* governance, not to governance: the pipeline in Section 4 runs deterministic checks on every call, consults a knowledge graph only when those checks are genuinely uncertain, and reaches a model only when the graph also has nothing — at which point the model's verdict can tighten the decision but can never loosen it.

The claim is deliberately narrow, in three respects. Vigil governs *tool calls*, not the agent's internal reasoning; a prompt injection that changes what the agent believes is out of scope until it manifests as a call. The graph consult adds real latency (measured in Section 8.3) and is therefore unconditional only for call shapes where the stakes warrant it. And when no inference vendor is configured, Vigil runs deterministic-only and says so — an uncertain verdict fails closed rather than being decorated with a synthesized risk score. We consider a governance layer that reports what it did not check more trustworthy than one that claims to have checked everything.

1.1 Contributions

- A deployed decision pipeline in which deterministic policy, graph context, cost forecasting, and behavioral analysis are composed ahead of any model, with the tighten-only invariant enforced by construction (Sections 3–4).
- Three graph-native checks over an extracted (never hand-built) knowledge graph: entity resolution of share recipients through SAME_AS alias chains, supply-chain blast radius for package installs, and behavioral-precedent lookup for anomalous tool patterns (Section 5).
- A vendor-failover inference chain with exhaustion-aware retirement and strictly downward role fallback, verified live against two of its three shipped vendors (Section 6).
- A tamper-evident audit construction covering every decision including allows, with an explicitly ordered hash preimage and an offline verifier (Section 7).
- An evaluation against real corpora and live services, reporting the numbers we measured — including the one that falls short of its target — and the eleven integration defects that only live testing surfaced (Sections 8–9).

2 Threat Model and Design Goals

We consider five principals: the *agent*, an untrusted process emitting tool calls through MCP; the *operator*, who declares session intent and budgets; the *firewall*, trusted for integrity of its own pipeline; the *graph* (HydraDB), trusted for retrieval but not for authority — its answers raise or resolve scrutiny, they do not settle anything alone; and the *model vendors*, untrusted for authority and unreliable for availability: any vendor may return wrong verdicts, malformed output, or exhaust its credit mid-session.

Goals. (G1) *Deterministic-first:* every call passes the deterministic checks; a call raising no signal completes with zero inference. (G2) *Tighten-only judgement:* a model verdict may escalate Allow to Pause or Block; no component may relax a deterministic Block. (G3) *Fail-closed uncertainty:* when neither policy nor graph nor model can adjudicate, the call does not proceed on a fabricated

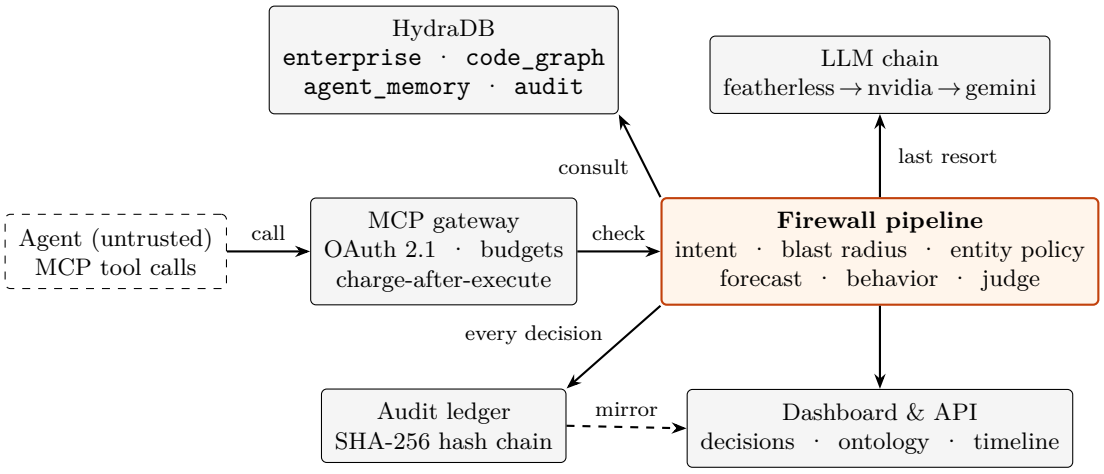

Figure 1: Vigil architecture. The shaded pipeline is the sole decision authority; the graph and the model chain are consulted, never obeyed — their outputs raise or resolve scrutiny inside the pipeline's own rules. The agent is untrusted and sees only the MCP surface.

verdict. (G4) *Tamper-evident totality:* every decision, including Allow, is appended to a hash chain whose verification requires no server. (G5) *Graceful degradation:* an absent graph credential or vendor key disables the corresponding consult and nothing else; the remaining pipeline is the same code, not a fallback mode.

Non-goals. Governing the agent's reasoning or context window; availability under a total outage of all three inference vendors *and* an operator policy that demands model adjudication (such calls fail closed, which is G3 doing its job); and resistance to an operator who edits the local ledger file and recomputes the entire chain — the ledger is tamper-*evident*, not tamper-proof (Section 10).

3 System Architecture

Vigil is built inside a SigNoz fork (Go 1.25); the governance layer is 17,022 lines under `pkg/query-service/vigil`, exposed through a single server binary. The MCP gateway performs OAuth 2.1 authorization, per-session budget accounting (charged after execution, so a blocked call costs nothing), a path jail on file tools, and shell execution disabled by default. Every `tools/call` passes through `firewall.Check` before its handler runs and through `Commit` after, so behavioral history reflects executed work rather than attempted work.

3.1 Why the graph is consulted before the model

A model asked "should this call proceed?" answers from priors. The graph answers from record: the enterprise collection knows which policies apply to which entity types because policy documents were ingested; `code_graph` knows the reverse-dependency closure of a package because 2,537 real npm packages were; `agent_memory` knows whether a tool pattern has precedent because every prior decision was logged into it. When the graph resolves a question — an extracted relationship, not a similarity guess — the decision is made without any model call. Only when the graph itself has nothing does the call escalate, which is both cheaper and harder to inject: a retrieval over ingested record has no instruction-following surface.

Table 1: Authority analysis. Each row is one pipeline component; each column an outcome it can produce. The final column is empty by construction — the code path that could relax a deterministic block does not exist.

Component	Final ALLOW	Final BLOCK	Escalate tier	Add context	Relax a BLOCK
Declared-intent policy	yes	yes	yes	—	—
Graph: intent adjudication	yes[a]	—	yes	yes	—
Graph: blast radius	—	yes	yes	yes	—
Graph: entity policy	—	yes	—	yes	—
Cost forecast	—	yes	yes	yes	—
Behavior engine	—	yes	yes	yes	—
Graph: behavioral precedent	—	—	—	yes	—
Model judge	—	yes	—	yes	—

[a]Only for a call the deterministic policy marked UNCERTAIN; a policy BLOCK is final before the graph is reached.

3.2 Why the model can only tighten

The tighten-only rule is not a convention; it is an absence. Deterministic blocks `return` from `Check` before the judge stage is reachable, so no verdict, however confident, can be consulted about them. The router's `RolesFor(TierNormal)` returns the empty set, so an unremarkable call cannot invoke a model even by accident. Both properties are asserted by tests that fail the build if a code change makes a model reachable on those paths.

4 The Decision Calculus

Let a call $c = (session, tool, args, cost)$ and let the decision lattice be

$$\textsc{Allow} \sqsubset \textsc{Pause} \sqsubset \textsc{Block}, \tag{1}$$

ordered by severity. Every stage may move the working decision upward in this order; goal G2 is precisely that no stage maps rightward-to-leftward. The pipeline runs six stages in a fixed order; Table 1 summarizes each stage's authority.

Stage 1: declared intent. The operator's session policy compiles to an allow/deny evaluation with deny-wins ordering; an allowlist miss yields UNCERTAIN rather than BLOCK, and an UNCERTAIN verdict is adjudicated against the enterprise graph ("what policy applies to this tool, and is it permitted?") before it is allowed to escalate toward a model.

Stage 2: blast radius, unconditional for install/exec-shaped calls. The package name is extracted heuristically (a scrutiny-raiser, not a parser) and three independent graph traversals run over `code_graph`: transitive reverse dependencies, maintainer overlap, and typosquat proximity. A typosquat finding blocks before any model is reachable; the block carries the literal extracted paths (`reqeusts --[is a typosquat of]--> requests`) so the audit record shows the graph's reasoning, not just its conclusion. An operator-declared compromised-package list is checked first and blocks immediately during an active incident.

Stage 2b: entity policy, unconditional for share/send-shaped calls. A recipient name n is resolved through the graph's extracted SAME_AS chain into an alias set $A(n)$, and an entity-type edge whose subject matches $A(n) \cup \{n\}$ is searched for a protected type t with an applicable

denial:

$$\text{fire} \iff \exists\,(s \xrightarrow{p} o):\ \text{match}(s, A(n) \cup \{n\}) \wedge t \in \text{lower}(p \,\|\, o)\ \wedge\ \text{deny} \in \text{policy paths}. \tag{2}$$

The subject-match requirement is load-bearing: without it, generic policy text ("policy no-pii-exfil applies to entity type Customer") flags *every* queried name, a defect we hit live before adding the constraint (Section 9).

Stage 3: cost forecast. With samples over a rolling window of length w minutes summing to S, remaining budget $R = B - C$, and horizon H:

$$\beta = S/w, \qquad \text{TTB} = R/\beta, \qquad P = C + \beta \cdot \min(H,\, 2\,\text{TTB}), \tag{3}$$

with BLOCK at $C \geq B$, and a soft state at $P \geq 0.8B$ that reroutes to a cheaper model role rather than killing an agent doing useful work. Every degenerate branch (no budget, no history, zero burn) returns a named state instead of a fabricated projection — the dashboard renders *insufficient history*, never an invented number.

Stage 4: behavior. Six detectors (loop, budget, retry storm, latency spike, timeout, stuck) evaluate the session's span history; a CRITICAL finding blocks, and any raised signal triggers a precedent lookup in `agent_memory` phrased over the *actual* accumulated pattern ("is `search_code` $\times$19, `network_request` $\times$3 anomalous, or does it match normal behavioral DNA?"). Three further detectors exist but read token counts and prompt text that MCP tool calls do not carry; they are not registered, and the rules endpoint reports exactly what is registered rather than advertising all nine (Section 10).

Stage 5: the gate. If no stage raised a signal, the call is allowed with `ModelUsed` empty. This is the majority path and it is inference-free by G1.

Stage 6: judgement. For tiers above Normal the router walks the role set (fast classifier, policy reasoner, security reviewer) over the vendor chain. The reply must survive `parseJudgment`: unknown fields rejected, all six fields required, enum membership, $0 \leq \text{risk} \leq 100$, $0 \leq \text{confidence} \leq 1$, one retry with the validation error appended, then a deterministic fail-closed default (block if any high-severity signal, else allow with the uncertainty recorded).

Abstention. Graph answers about a subject s are gated by content presence rather than score. With $K(s)$ the significant words of s and T the full retrieved text:

$$\text{abstain}(s) \iff 2 \cdot \left|\{k \in K(s) : k \in T\}\right| \ \leq\ |K(s)|, \tag{4}$$

i.e. a majority of the subject's own words must appear in what was retrieved. Both halves of the conventional check — empty graph context, or a confidence threshold — were tested directly against a fabricated subject and failed to discriminate: HydraDB returned ten chunks scored 0.87–0.90 *and* fifteen extracted triplets for an entity that has never existed anywhere in the corpus (Section 9, finding 5).

5 Graph-Native Context

HydraDB is a managed graph-plus-memory store whose defining property, for this design, is that the graph is *extracted from ingested plain text* — entities, relationships, and canonical predicates are produced server-side from English sentences. Vigil never writes an edge. The entity resolver (Section 8.1) emits its findings as sentences (" 'Sam' is the same person as 'Soham Ratnaparkhi',

resolved with confidence 0.97 via deterministic email match ") and HydraDB extracts `sam --[also known as]--> soham ratnaparkhi`. This discipline has a cost — the extractor's canonical vocabulary must be discovered empirically, not assumed (Section 9) — and a benefit: every fact in the graph is traceable to an ingested document, so the graph cannot silently contain anything no one wrote.

Four collections partition the context by question: `enterprise` (720 documents: the ontology corpus, policies, resolver output), `code_graph` (2,537 real npm packages ingested from the public registry and deps.dev — no fixtures), `agent_memory` (398 memory records: conversation sessions, behavioral baselines, and every firewall decision, fire-and-forget with a token-bucket rate limiter), and `audit` (141 mirrored decisions, making the audit trail graph-queryable: "show every block that traces to this package" is a traversal, not a scan).

6 Failover Inference

The vendor table lists Featherless (the hackathon's compute partner, first in priority), NVIDIA NIM, and Gemini via its OpenAI-compatibility layer — three real shipped vendors over one generic client, not one vendor and stand-ins. `Chain.Complete` tries vendors in order; a vendor returning quota exhaustion or auth failure is *retired* for the process lifetime (subsequent calls skip it without paying its latency), while transient errors move on without retiring. Within a vendor, role fallback is strictly downward — reviewer to reasoner to fast — because a cheap model's outage silently invoking the expensive one converts a failure into a bill. A boot-time probe calls each vendor once on its cheapest role, so "live" on the status endpoint means *answered*, not *configured*. At the time of writing no Featherless credential exists; the chain runs `nvidia→gemini`, verified by a live test over the exact constructor the server uses, and a Featherless key slots back in first with no code change.

7 Verification and Audit

Every decision — including every Allow, since a trail recording only blocks proves nothing about what got through — is appended to a JSONL ledger. The hash preimage is explicitly ordered and separated by the ASCII record separator `0x1E` (which cannot occur in any hashed value, so no value can impersonate a field boundary):

$$h_k = \text{SHA-256}\big(id \parallel ts \parallel agent \parallel session \parallel tool \parallel H(args) \parallel decision \parallel reason \parallel model \parallel cost \parallel h_{k-1}\big), \tag{5}$$

deliberately *not* a hash of the JSON encoding, whose field order is stable only by convention and whose future extension would silently invalidate untouched historical chains. Arguments are hashed rather than stored: an audit log is the wrong place to accumulate a copy of every path and command an agent ever touched, and the hash still proves two calls were identical, which is what loop and replay detection need. `vigil audit verify` recomputes the chain offline — tampering, reordering, or deletion is reported at the first divergent index — and the live ledger holds 355 real events at the time of writing.

Table 2: Checks applied to a model verdict and the failure each forecloses. All must hold or the deterministic fail-closed default applies.

Check	Forecloses
Unknown JSON fields rejected	Smuggled directives riding a valid-looking verdict
All six fields required	A missing `decision` read as "default allow"
Enum membership on `decision`/`severity`	Free-text verdicts interpreted charitably
$0 \leq$ risk ≤ 100, $0 \leq$ confidence ≤ 1	Out-of-range scores skewing downstream thresholds
Exactly one retry, with the validation error	Unbounded re-prompting of a failing vendor
Verdict applied through the lattice only	Any model output relaxing a deterministic block

8 Implementation and Evaluation

The implementation comprises the Go governance layer (17,022 lines across firewall, graph client, LLM chain, audit, policy, behavioral engine, recovery, MCP, and OAuth packages), a Next.js 16 dashboard (mission control, ontology, blast radius, memory timeline, cost firewall, models), and three Python corpus tools. 129 tests pass across 22 files; four suites run against live services — the graph client's integration suite, the vendor chain, entity policy, and behavior DNA — gated on credentials so the default build stays offline.

8.1 Entity resolution (enterprise ontology)

A simulated nine-source corpus (Slack, Gmail, Linear, Drive, HubSpot, Fireflies, GitHub, Jira, Confluence) of 500 documents for a fictional company was generated — simulation disclosed and precedented (EnterpriseRAG-Bench's fictional-company pattern), since no real enterprise export was available — with deliberately planted name-variant drift and 66 contradiction pairs. The three-stage resolver produced 54 SAME_AS proposals: 35 by deterministic email match (confidence 1.0), 19 by name-similarity heuristic, 0 requiring the LLM-assist stage. The end-to-end proof is a live firewall test, not a stub: "share the export with Jordan" BLOCKS — Jordan resolves through `jblake/jordan acme/jordan blake` to the corpus's one Customer, whose type a policy denies — while "share the export with Sam" ALLOWS, Sam resolving to an Employee. Abstention on invented names measured 15/15 by content presence.

8.2 Memory and temporal reasoning (LongMemEval)

Real data: the `longmemeval-cleaned` oracle split (ICLR 2025), 41 sessions sampled across all six question categories, ingested in 240 chunks with zero failures, plus four FACT_UPDATE resolver statements giving knowledge-update questions an explicit superseded-by chain. Table 3 reports retrieval recall — does the answer's own vocabulary surface in what the graph returned for the real question — which is the property Vigil's abstention layer and downstream consumers actually depend on; it is not an LLM-graded answer-quality score and is not presented as one.

8.3 Latency

Broken out by query shape rather than averaged into one flattering number: plain memory retrieval measured 220–260 ms; graph-context queries (what blast radius and entity policy use)

Table 3: LongMemEval oracle evaluation, as measured. "Skipped" answers had no checkable keyword (bare numerals). The abstention figure is reported against its 95% design target, not adjusted toward it.

Category	Recall	n (skipped)
single-session-user	100.00%	4 (0)
single-session-assistant	100.00%	4 (0)
single-session-preference	100.00%	4 (0)
knowledge-update	100.00%	2 (2)
temporal-reasoning	100.00%	4 (0)
multi-session	50.00%	2 (2)
Overall	**95.00%**	**20 scored**
Abstention accuracy (held-out)	56.25%	16

measured 375 ms–1 s isolated and up to 4–5 s under concurrent ingest load; HydraDB's deeper `thinking` mode, 2.5–5 s, is not used on the decision path. A full entity-policy consult (five graph queries) measured ∼3.6 s — material, and the reason graph consults are unconditional only for install- and share-shaped calls rather than for everything. Live vendor probes at boot: NVIDIA 434 ms, Gemini 1.1 s.

9 Findings from Testing Against Live Services

Every integration in this system was verified against the running service rather than its documentation, and the method paid for itself eleven times. We report these because each one would have shipped as a silent correctness failure under documentation-driven development.

1. **Predicate vocabulary is discovered, not assumed.** HydraDB canonicalizes identity statements to `also known as` (not "same person") and denials to the present-tense `denies` — not a substring of "denied", so a deny-keyword check matching `denied/forbid` silently never fired on real output. This produced an actual false ALLOW in a live test before being caught.
2. **Compound questions lose to the ranker.** Asking for aliases and policy in one query let the more numerous policy documents crowd the identity triplets out of retrieval entirely; each profile question now asks for exactly one kind of thing.
3. **Alias-shaped queries retrieve everyone's aliases.** Structurally similar identity sentences co-retrieve; a triplet counts as an alias only if its subject or object names the queried entity.
4. **Catalog listings are not entitlements.** NVIDIA's `/v1/models` lists models the account cannot invoke (two 404'd with "Function not found for account"); every shipped model ID was confirmed with a real completion. Gemini's free tier blocks `-pro` models outright (429, stated limit 0/day) and its plain `-flash` models spend hidden thinking tokens before any visible content — `-flash-lite` exhibits neither problem and serves all three roles on this key.
5. **Relevancy score is rank order, not relevance.** A query about a fabricated entity still returns ten chunks scored 0.87–0.90; abstention must check content presence (Eq. 4), and a single-keyword match is itself insufficient for multi-word subjects — majority presence is required.

6. **Ingest limits are token-denominated.** A 413 on a 19 kB session decoded to "request cost 3900 exceeds the per-request per_sec limit of 1000"; plain repeated characters up to 30 kB ingested fine. Sessions are chunked to ∼3200 characters and paced with retry-on-429.
7. **Response shape varies by ingest type.** `chunk_content` is JSON-wrapped for knowledge sources and raw text for memory sources; a parser assuming one shape silently dropped the top-ranked chunk of its own query.
8. **Metadata that is never ingested does not exist.** An `entity_type` tag on the resolver's cast list was dead data until emitted as a sentence per person; before that, every queried name matched the same generic policy text identically.
9. **Behavioral queries must speak the baseline's vocabulary.** "Has this pattern been seen before" lost to decision-log noise on the shared collection; "is this pattern *anomalous…*" — echoing the ingested baseline's own wording — retrieved it reliably, and the production query now also embeds the real accumulated pattern rather than abstract signal names.

10 Trust Assumptions and Limitations

Stated plainly, in decreasing order of weight.

1. **Keyword adjudication of extracted predicates.** Deny/anomaly/entity-type detection is substring matching over free-form extracted English. It is deliberately conservative (ambiguity does not block) and empirically tuned (Section 9), but it is a heuristic, and finding 1 shows exactly how such heuristics fail.
2. **Abstention accuracy is 56.25% against a 95% target.** The keyword-majority gate genuinely does not reach the target on diverse natural-language questions — some held-out questions share enough real vocabulary with unrelated ingested content that the graph answers when it should abstain. We report the measured number.
3. **The ontology corpus is simulated.** Disclosed, precedented, and structurally realistic, but a resolver validated on synthetic name drift has not been validated on the messier drift of a real enterprise. The memory corpus, by contrast, is real benchmark data — though the oracle split, which omits the padding haystack sessions of the full 115k-token benchmark, a disclosed scope reduction.
4. **Featherless is configured but unproven.** The chain is verified live on NVIDIA and Gemini over the identical wire format and code path; the specific first-priority vendor awaits a credential. The claim is "the chain works and Featherless slots in," not "Featherless works."
5. **Graph latency on the hot path.** 375 ms–1 s per graph-context query, degrading under load, bounds how many call shapes can justify an unconditional consult. Timeouts degrade the consult, never hang the call.
6. **Six of nine detectors.** Token-explosion, repeated-prompt, and prompt-recursion detectors need LLM-turn data that tool-call interception does not carry; they are unregistered and unadvertised rather than falsely enabled.
7. **Injection scope.** Vigil adjudicates tool calls. A compromised agent that never emits a governable call is invisible to it; a compromised agent that does emit one meets every stage above.

8. **The ledger is tamper-evident, not tamper-proof.** A host-level adversary can rewrite the file and recompute the chain; the mirror in HydraDB makes this detectable, not impossible.

11 Related Work

Content guardrails (NeMo Guardrails, LlamaFirewall-style systems) interpose on the conversation — inputs, outputs, retrievals — and are complementary: Vigil governs the side-effect boundary they do not reach, and does so with a deterministic-first pipeline rather than classifier-first. **Observability platforms** (LangSmith, Langfuse, and the SigNoz lineage this repository forks) record what agents did; Vigil sits upstream with authority to refuse, and keeps the observability — every decision is a span. **Policy engines** (OPA/Rego-style) supply the deterministic stage in principle but have no notion of graph-resolved identity, spend trajectory, or behavioral precedent; Vigil's stages 2–6 are what a policy engine cannot express. **Agent memory systems** (Mem0, Zep) optimize recall for the agent's benefit; Vigil turns the same memory substrate against the agent's failure modes — precedent lookup, temporal supersession, and audit mirroring — and evaluates on the same public benchmark (LongMemEval) those systems compete on.

12 Future Work

Closing the abstention gap with a learned or graph-structural gate rather than keyword majority; proving the Featherless leg live; extending unconditional graph consults as latency permits, with caching for repeated subjects; ingesting a real (consented) enterprise corpus to validate the resolver beyond synthetic drift; LLM-turn interception to activate the three dormant detectors; and anchoring periodic ledger checkpoints externally so tamper-evidence survives a host-level adversary.

13 Conclusion

A tool call is a side effect with a model attached, and the model should not be the last authority on its own side effects. Vigil makes the authority structural: deterministic rules that always run, a graph that answers from record rather than prior, a model that is reached last and can only tighten, and an audit chain that remembers every outcome including the boring ones. The composition rests on one discipline applied throughout — against the graph, against the vendors, against our own claims: *verify against the live system, and report what was measured.* Eleven integration defects, one honest 56.25%, and every number in this paper exist because of it.

References

[1] HydraDB. *Managed Graph + Memory API, v2: Ingestion, Query, and Graph Context.* docs.hydradb.com, 2026.

[2] Wu, D. et al. *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory.* ICLR 2025; community `longmemeval-cleaned` release, 2025.

[3] Anthropic. *Model Context Protocol Specification.* modelcontextprotocol.io, 2025.

[4] Hardt, D. et al. *OAuth 2.1 Authorization Framework.* IETF draft, 2024.

[5] Rebedea, T. et al. *NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications.* EMNLP Demo, 2023.

[6] Open Policy Agent. *Policy-Based Control for Cloud-Native Environments.* openpolicyagent.org.

[7] Mem0. *The Memory Layer for AI Agents.* mem0.ai, 2025.

[8] SigNoz. *Open-Source Observability Platform.* signoz.io. (Upstream of this fork.)

[9] Project Vigil. *Source, live tests, and corpus tooling.* github.com/Aaditya1273/Vigil, 2026.

Vigil is hackathon software. Evaluation numbers in this paper were measured against the deployed HydraDB database `vigil-os` and the live NVIDIA and Gemini endpoints on August 19–20, 2026, by the test suites named in the text; the formulas correspond to the shipped evaluation code in `pkg/query-service/vigil/firewall` and `pkg/query-service/vigil/audit` at the commit tagged for submission. The Track 01 corpus is simulated and labeled as such; the Track 03 corpus is real benchmark data under its oracle split. Nothing in this paper claims a result that was not measured.